

deep learning framework written in Julia by Dr Deniz Yuret along with other contributors (<https://github.com/denizyuret/Knet.jl>).

The Knet framework supports GPU operations. It has support for automatic differentiation, which is carried out using dynamic computational graphs.

Similar to Flux, you can start using Knet directly without any major installation process. You need to simply add it with `Pkg.add()` as shown below:

```
Pkg.add("Knet")
```

If you encounter any installation issues, follow the instructions given in the official Web page at <http://denizyuret.github.io/Knet.jl/latest/install.html>.

A simple example of using Knet for training and testing the LeNet model for the MNIST handwritten digit recognition data set is shown below (source: <https://github.com/denizyuret/Knet.jl>):

```
-
using Knet

# Define convolutional layer:
struct Conv; w; b; f; end
(c::Conv)(x) = c.f.(pool(conv4(c.w, x) .+ c.b))
Conv(w1,w2,cx,cy,f=relu) = Conv(param(w1,w2,cx,cy),
param0(1,1,cy,1), f)

# Define dense layer:
struct Dense; w; b; f; end
(d::Dense)(x) = d.f.(d.w * mat(x) .+ d.b)
Dense(i::Int,o::Int,f=relu) = Dense(param(o,i), param0(o), f)

# Define a chain of layers:
struct Chain; layers; end
(c::Chain)(x) = (for l in c.layers; x = l(x); end; x)

# Define the LeNet model
LeNet = Chain((Conv(5,5,1,20), Conv(5,5,20,50),
Dense(800,500), Dense(500,10,identity)))

# Train and test LeNet (about 30 secs on a gpu to reach 99%
accuracy)
include(Knet.dir("data","mnist.jl"))
dtrn, dtst = mnistdata()
train!(LeNet, dtrn)
accuracy(LeNet, dtst)
```

Knet documentation has detailed instructions for handling major deep learning building blocks, some of which are listed below:

- Back propagation
- Convolutional Neural Networks

- RNN
- Reinforcement learning

If you are a researcher working in the deep learning domain, you may find the paper titled ‘Knet: Beginning deep learning with 100 lines of Julia’ by Dr Deniz Yuret interesting.

MLBase.jl

Unlike the previous two libraries, the MLBase.jl library doesn’t implement specific algorithms used in ML. But it provides a number of handy tools to assist building ML programs. The MLBase.jl may be used for the following tasks:

- Preprocessing and data manipulation
- Performance evaluation
- Cross-validation
- Model tuning

MLBase.jl depends on the StatsBase package. Detailed usage instructions are available at the official documentation page at <https://mlbasejl.readthedocs.io/en/latest/index.html>.

TensorFlow.jl

TensorFlow.jl is a wrapper around the most popular ML framework from Google — TensorFlow.

TensorFlow.jl can be added using `Pkg.add()` as shown below:

```
Pkg.add("TensorFlow")
```

A basic usage example is shown below:

```
using TensorFlow
using Test

sess = TensorFlow.Session()

x = TensorFlow.constant(Float64[1,2])
y = TensorFlow.Variable(Float64[3,4])
z = TensorFlow.placeholder(Float64)

w = exp(x + z + -y)

run(sess, TensorFlow.global_variables_initializer())
res = run(sess, w, Dict{z=>Float64[1,2]})
@test res[1] ≈ exp(-1)
```

If you are a machine learning researcher, then you may find the paper titled ‘TensorFlow.jl: An Idiomatic Julia Front End for TensorFlow’ by Malmaud and White very interesting. This paper has been published in the *Journal of Open Source Software*.

If you have worked with TensorFlow in Python, you may find the Julia version very similar. There are some inherent advantages of the Julia interface over Python which are illustrated in detail in the official documentation at https://github.com/malmaud/TensorFlow.jl/blob/master/docs/src/why_julia.md.